# CAN-Sentinel

## Technical Guide: How the Automotive ECU Intrusion Detection Demo Works

**Project:** CAN-Sentinel: Automotive ECU Intrusion Detection System
**Audience:** Security Admin Associate L1 / Cyber Security internship
**Scope:** Offline CAN 2.0B analysis and synthetic attack simulation

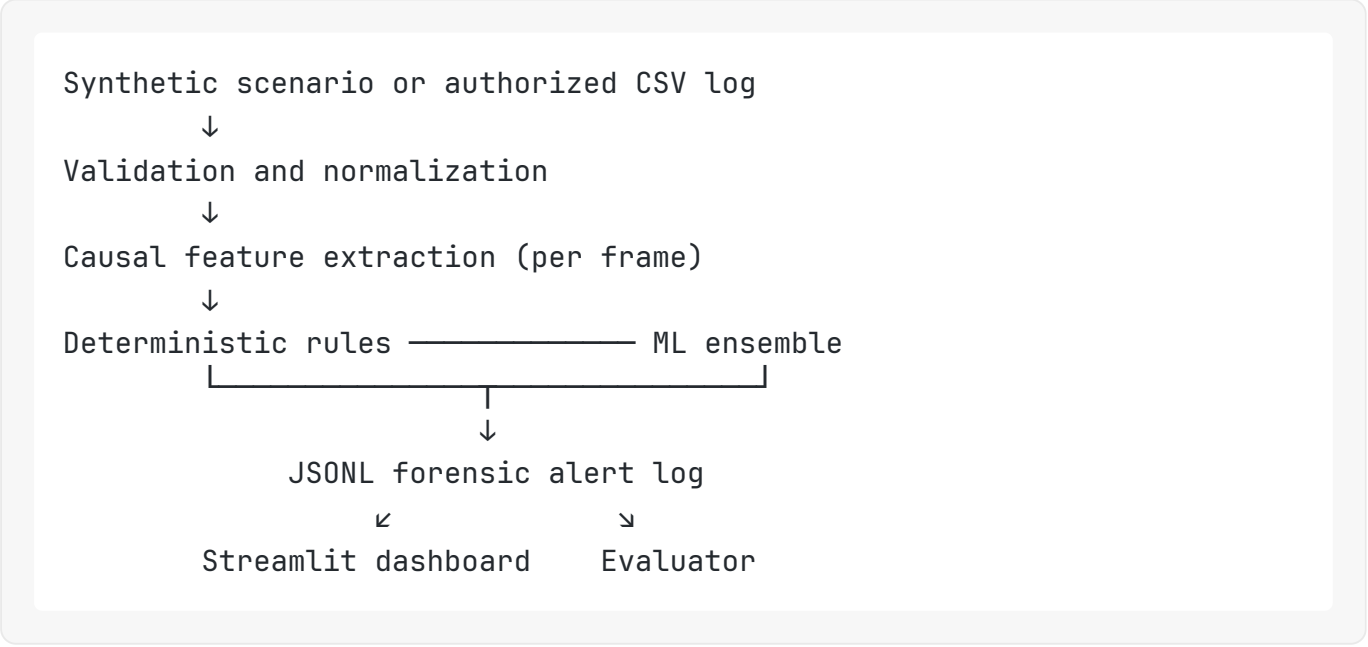
---

## 1. Executive overview

CAN-Sentinel is a Python demonstration of a small intrusion-detection pipeline for Controller Area Network (CAN) traffic. It generates repeatable examples of benign traffic and three attack patterns, validates and parses packet records, computes causal per-frame features, evaluates deterministic checks and statistical classifiers, records alerts as JSON Lines, and presents analysis in a Streamlit dashboard.

The implementation is intentionally simulation-first. It does not open a CAN adapter, interact with a vehicle, or transmit frames. Its model and thresholds are demonstrators, not validated protection for any particular ECU, vehicle, or safety function. In particular, synthetic benchmark results cannot establish production detection capability.

At a high level:



## 2. CAN and the security problem

CAN is a message-oriented bus used by in-vehicle electronic control units. A frame carries an arbitration identifier, a data-length code (DLC), and a payload. Classical CAN supports payloads up to eight bytes. Arbitration gives lower numeric identifiers higher priority when simultaneous senders contend for the bus.

The identifier is not a cryptographic sender identity. On a conventional CAN segment, a node able to transmit may be able to send a frame using another ECU's identifier. High-priority traffic can also consume bus capacity and delay lower-priority messages. Therefore, IDS monitoring commonly considers message identity and expected timing, payload patterns, and rate anomalies.

CAN-Sentinel's sample rules and synthetic attack signals illustrate those concepts. They do not implement authentication, bus arbitration timing analysis at the electrical layer, an OEM signal database, or safe automated response.

## 3. Repository structure

```
CAN-Sentinel/
├── app.py                          Streamlit dashboard
├── generate_report.py              Markdown and optional PDF report genera
├── requirements.txt                Runtime dependencies
├── requirements-dev.txt            Runtime dependencies plus pytest
├── README.md                       Installation and project overview
├── CAN-Sentinel-Project-Report.md Vulnerability summary
├── CAN-Sentinel-Technical-Guide.md This walkthrough
├── can_sentinel/
│   ├── __init__.py                 Package version
│   ├── simulator.py                Typed frame model and traffic generat
│   ├── preprocessor.py             CSV validation and causal features
│   ├── detector_rules.py            Deterministic rate/ID/DLC checks
│   ├── detector_ml.py               Isolation Forest + Random Forest
│   ├── engine.py                    Pipeline, alert fusion, JSONL loggin
│   └── evaluator.py                 Metrics and latency summary
└── tests/
    └── test_pipeline.py             Unit/smoke tests
```

## 4. Data model and traffic generation

`can_sentinel/simulator.py` defines the immutable `CANFrame` record: timestamp, arbitration ID, DLC, payload bytes, label, and scenario name. Construction checks that the ID is in the standard/extended CAN identifier range, the DLC is 0–8 and matches payload size, and timestamps are non-negative.

`TrafficSimulator(seed=...)` uses a private seeded pseudorandom generator. Reusing the same seed and method arguments produces the same synthetic sequence. This supports repeatable demonstrations and regression tests; it does not model every timing, signal, or physical property of a vehicle network.

The supported scenarios are:

- **Normal:** cyclic identifiers ( `0x100` , `0x120` , `0x200` , `0x316` , `0x400` ) and eight-byte payloads. The example `0x316` engine-speed field uses the first two bytes as little-endian RPM values in a modest range.
- **DoS:** repeated `0x000` frames. The default interval is one tenth of the normal period, hence ten times the configured baseline frame rate for that generator. The identifier is also configured as reserved in the default rule detector, so this scenario triggers a deterministic rule.
- **Fuzzy:** random standard identifiers from `0x000` through `0x7FF` and random eight-byte payloads.
- **Spoof:** frames using engine-speed identifier `0x316` with first two bytes encoding implausible 7000–9999 RPM values.

`scenario()` concatenates the normal sequence and attack sequence, assigning ground-truth label 0 to normal and 1 to injected attack frames, then sorts by timestamp.

## 5. Input validation and feature extraction

`can_sentinel/preprocessor.py` accepts CSV data with required columns `Timestamp` , `Arbitration_ID` , `DLC` , and `Data_Bytes` . It accepts decimal IDs or `0x` -prefixed identifiers. Hex payload text may contain whitespace and common separators. Parsing checks required columns, valid numeric ranges, DLC limits, payload length agreement, and timestamp ordering. Out-of-order rows are stably sorted by timestamp.

`FeatureExtractor` is stateful: it processes records in order and only uses current and prior observations. That causal property avoids using future frames when the same extractor is used in an online pipeline. The feature vector is:

| Feature | Meaning |
| --- | --- |
| `iat` | Time since the previous observed frame (seconds; first frame is 0) |
| `id_frequency` | Occurrences of the current arbitration ID in the recent frame window, including this frame |
| `payload_entropy` | Shannon entropy of payload byte values; 0 for an empty payload |
| `dlc` | Data length code |
| `identifier` | Numeric arbitration ID |
| `id_novelty` | 1 if the ID has not appeared earlier in this extractor's session, otherwise 0 |
| `rpm_raw` | Little-endian first two payload bytes for ID `0x316`; otherwise 0 |

The recent ID frequency uses a bounded window. ID novelty is session-cumulative. These features are illustrative. A real detector normally requires signal definitions, vehicle-specific baselines, data-quality handling, clock analysis, and validated train/test separation.

## 6. Deterministic rule detector

`can_sentinel/detector_rules.py` applies interpretable checks. The default configuration has a one-second rolling time window, a total bus rate threshold, a per-ID threshold, and reserved identifier set containing `0x000`.

- **DLC bounds:** flags values outside 0–8.
- **Reserved ID:** flags any observed ID in configured reserved IDs.
- **Bus flood:** flags when observed frames in the time window exceed the aggregate threshold.
- **Per-ID rate:** flags when the current identifier exceeds its configured rate threshold.

Thresholds are demonstration defaults. A legitimate vehicle can use an ID that is reserved by this demo, or exceed a demo rate limit. Production rules must be derived from authorized vehicle-specific specifications and benign traces, with careful false-positive analysis.

## 7. Statistical / machine-learning detector

`can_sentinel/detector_ml.py` trains two scikit-learn estimators:

1. **Isolation Forest** learns a representation of generated benign feature vectors and returns an anomaly decision score.
2. **Random Forest classifier** is trained on generated benign and labeled attack feature vectors (DoS, fuzzy, spoof).

The detector combines the supervised class probability and a squashed Isolation Forest score into one demonstration anomaly score. A configurable decision threshold determines whether the pipeline emits an ML anomaly alert. The random seed, tree counts, and training generator are fixed/configurable for repeatability.

Important interpretation: the Random Forest has seen examples from the same three synthetic attack families it is later intended to identify. This is not a rigorous zero-day evaluation. An Isolation Forest is not a guarantee of detecting unknown attacks. A production-quality assessment needs independent data splits—ideally holding out vehicles, routes, sessions, or attack families—plus calibration, drift analysis, class imbalance handling, and analyst review.

## 8. Real-time-compatible pipeline and forensic events

`can_sentinel/engine.py` couples rule checks and ML inference. For each input frame it:

1. validates identifier, DLC, payload size, and timestamp;
2. obtains deterministic rule alerts;
3. extracts the current causal feature vector;
4. computes the ML anomaly score when a fitted model is supplied;
5. measures processing duration using a monotonic high-resolution timer;
6. creates alert records and appends them as JSON Lines when any detector fires.

Each alert records UTC event time, frame timestamp, arbitration ID, severity, rule name, explanatory detail, ML score, and observed processing duration. JSONL is easy to stream and inspect, but this prototype does not provide cryptographic integrity, tamper-evident storage, log rotation, retention policy, access control, or a trusted vehicle clock. Those are important forensic deployment requirements.

The engine is compatible with per-frame processing, but ordinary Python and a general-purpose OS are not hard-real-time. Inference can take several milliseconds or more depending on system load, model configuration, and installed library versions. The project must not claim a sub-millisecond guarantee. Measure on the target hardware and design bounded queues, overload behavior, and failure handling before any operational use.

## 9. Evaluation metrics

`can_sentinel/evaluator.py` accepts labeled frames, processes each frame once, and reports:

- **Accuracy:** `(TP + TN) / all frames`.
- **Precision:** `TP / (TP + FP)`; among alerts, fraction truly malicious.
- **Recall:** `TP / (TP + FN)`; fraction of malicious frames detected.
- **F1:** harmonic mean of precision and recall.
- **False-positive rate:** `FP / (FP + TN)`.
- **ROC-AUC:** ranking ability across score thresholds; only defined when both labels are present.
- **Mean and p95 latency:** observed per-frame processing duration in milliseconds on the current host.
- **Confusion matrix:** true negatives, false positives, false negatives, and true positives.

Metrics are threshold-, data-, and host-dependent. A class-imbalanced dataset can make accuracy misleading. At vehicle scale, event-level metrics, detection delay, per-attack breakdown, false alarms per operating hour, resource usage, and safe-response impact should also be considered. Synthetic demo scores are not product claims.

## 10. Streamlit dashboard walkthrough

Launch from the repository root:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

In the sidebar, select an attack (`dos`, `fuzzy`, or `spoof`), normal/attack frame counts, and a random seed. Press **Run simulation**. The app creates synthetic data, trains the demonstration model, runs the IDS, evaluates the sequence, and stores results in Streamlit session state.

The dashboard presents KPI values, traffic timestamp versus arbitration ID, the most recent packet records, a ROC curve when both labels exist, and the recent deterministic/ML alert

records. For a useful live-looking replay, change the seed and rerun; this dashboard performs a bounded batch simulation rather than subscribing to a physical CAN interface.

## 11. Reports, tests, and reproducibility

Generate the vulnerability summary in Markdown or PDF:

```
python generate_report.py
python generate_report.py --pdf
```

Run test dependencies and checks:

```
pip install -r requirements-dev.txt
python -m pytest -q
python -m compileall -q .
```

The tests cover seeded simulation, frame and CSV input validation, reserved-ID alert creation/JSONL output, and metric calculation. Tests are smoke/unit checks and do not establish security efficacy, timing guarantees, or safety suitability.

## 12. Vulnerability and deployment considerations

| Concern | Why it matters | Safer next step |
|---|---|---|
| Missing CAN sender authentication | A malicious/compromised node can impersonate IDs | Gateway segmentation and access policy; assess CANsec where supported |
| Availability / arbitration abuse | Priority traffic may delay critical frames | Validate deadlines and fail-safe behavior; gateway rate controls |
| Fixed demo thresholds | Legitimate vehicle schedules vary | Use OEM-approved allowlists and representative trace baselines |
| Synthetic model data | Does not represent real fleet variability | Gather consented, labeled data; perform leakage-safe holdouts |
| Alert log integrity | Local JSONL can be edited or lost | Protected storage, signed exports, time synchronization, retention controls |
| No automated response | Alerts do not isolate or remediate a compromised ECU | Keep human/safety governance; any response requires hazard analysis |
| Runtime variability | Python host timing varies | Profile on target, bound operations, review production implementation |

## 13. Safe-use statement

Use only synthetic examples or vehicle data that you are authorized to inspect. Do not connect the demo to public-road vehicles or transmit crafted CAN frames. Any future interface to a real bus needs explicit authorization, isolated hardware-in-the-loop testing, vehicle safety approval, and a separate security and hazard review. CAN-Sentinel is an educational prototype and is not safety-certified or production-ready.